API RATE LIMITING & SECURITY GATEWAY

Hands-On Cloud Security Project using Azure API Management

OBJECTIVE	Protect APIs using native Azure cloud security mechanisms — no custom code, entirely configured through the Azure Portal UI.

API Gateway	Token Auth	Rate Limiting	Request Logging	Access Policies
Azure APIM	Subscription Keys	10 req / 60 sec	App Insights	IP Filtering

■ **Author:** Yashraj	■ **Field:** Computer Engineering — 1st Year	■ **Date:** May 3, 2026
■■ **Platform:** Microsoft Azure	■■ **Service:** API Management (APIM)	■ **Tier:** Developer (No SLA)

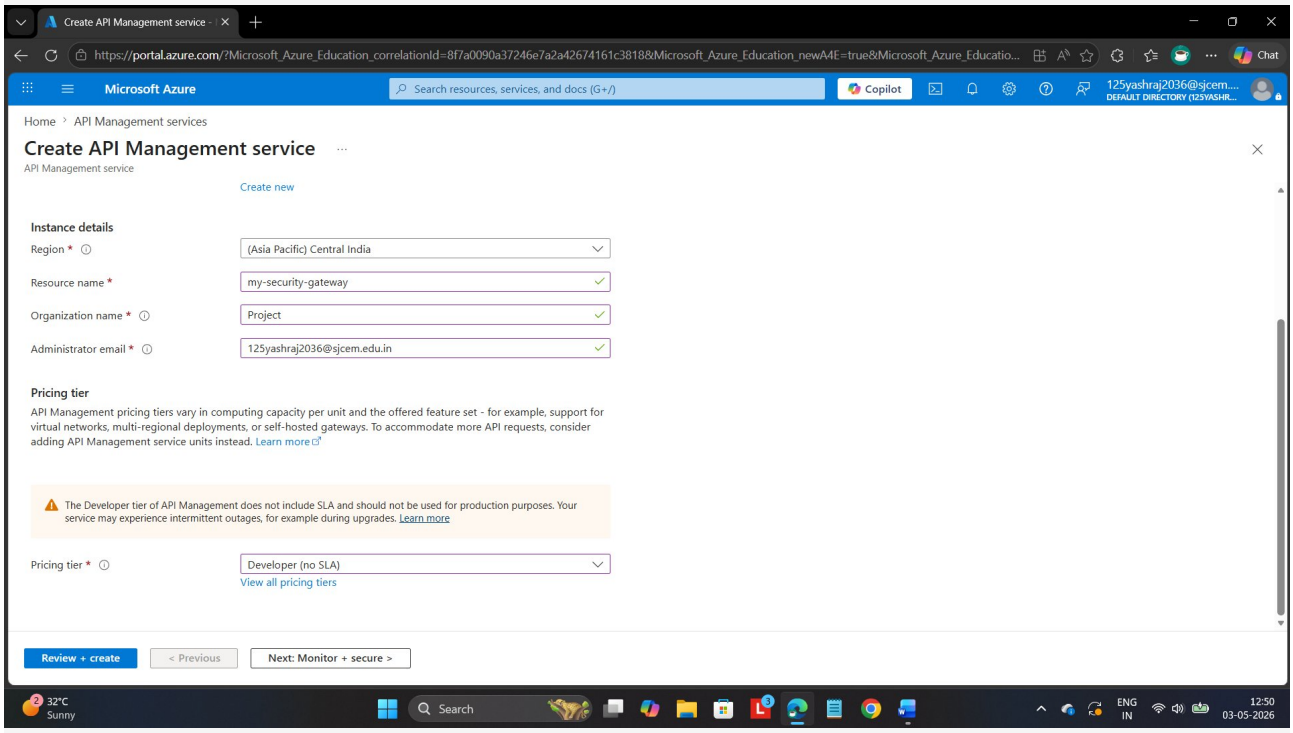
Project Results at a Glance

Test Result	Observation
200 OK ■	API Gateway live & routing — https://my-security-gateway.azure-api.net
401 Denied ■	Wrong subscription key rejected instantly by the gateway
429 Rate Limit ■	11th request blocked — retry-after header returned
403 Forbidden ■	IP filter policy blocked request from unlisted IP range
Logging Live ■	Application Insights capturing all requests with full metadata

Step-by-Step Implementation

The following screenshots document the complete setup of the API Rate Limiting & Security Gateway on Microsoft Azure using the Azure Portal UI. Each step demonstrates one or more of the five security guidelines.

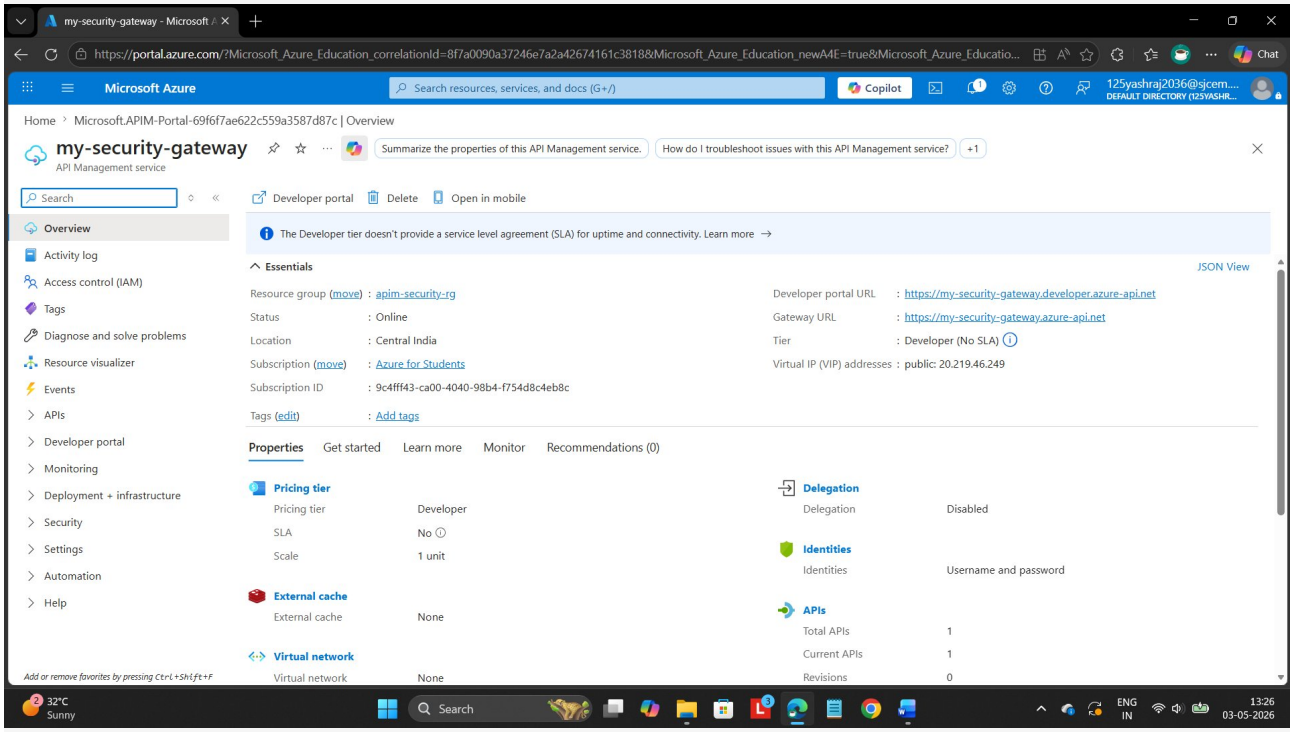
Step 1 | Create API Management Service



APIM creation form — Resource: my-security-gateway | Tier: Developer (No SLA) | Region: Central India

■ APIM instance configured and submitted for deployment. Deployment takes 10–15 minutes.

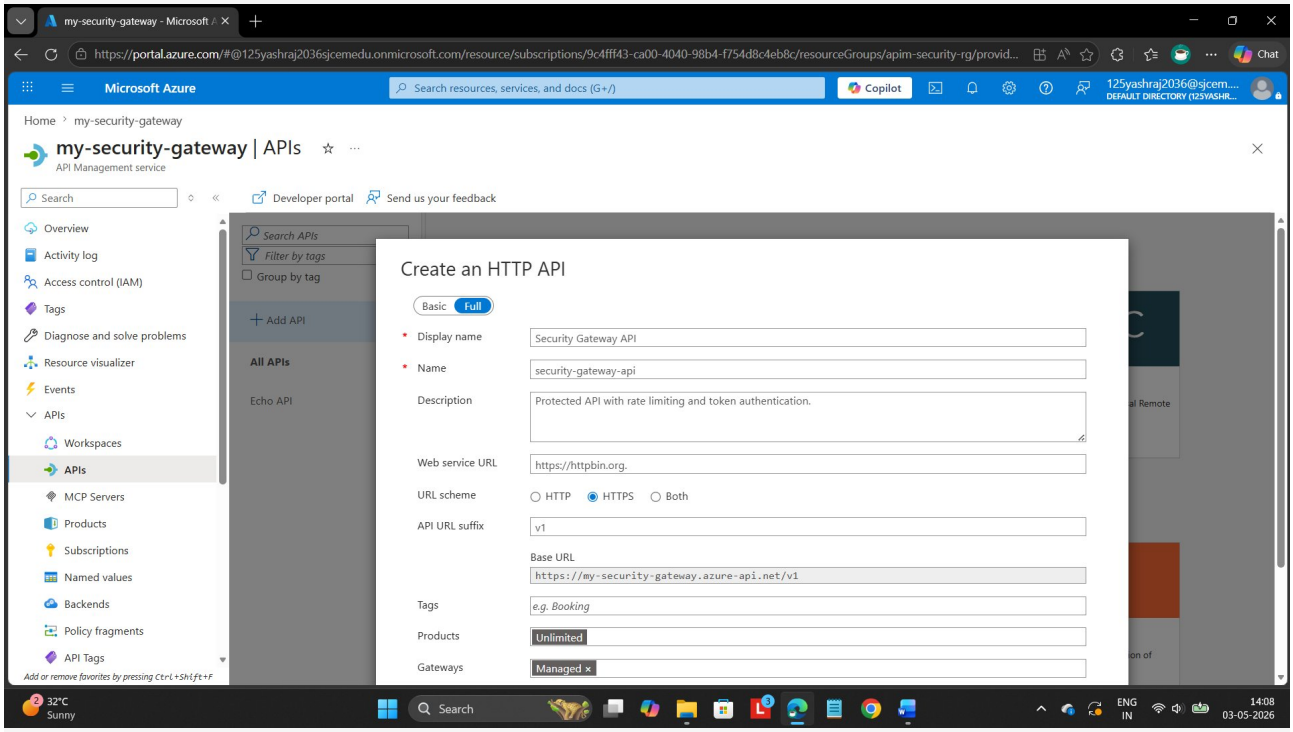
Step 2 | APIM Instance Live



APIM Overview — Gateway URL: https://my-security-gateway.azure-api.net | Status: Online

■ API Gateway is live. All traffic to the gateway URL is intercepted and security-checked before reaching the backend.

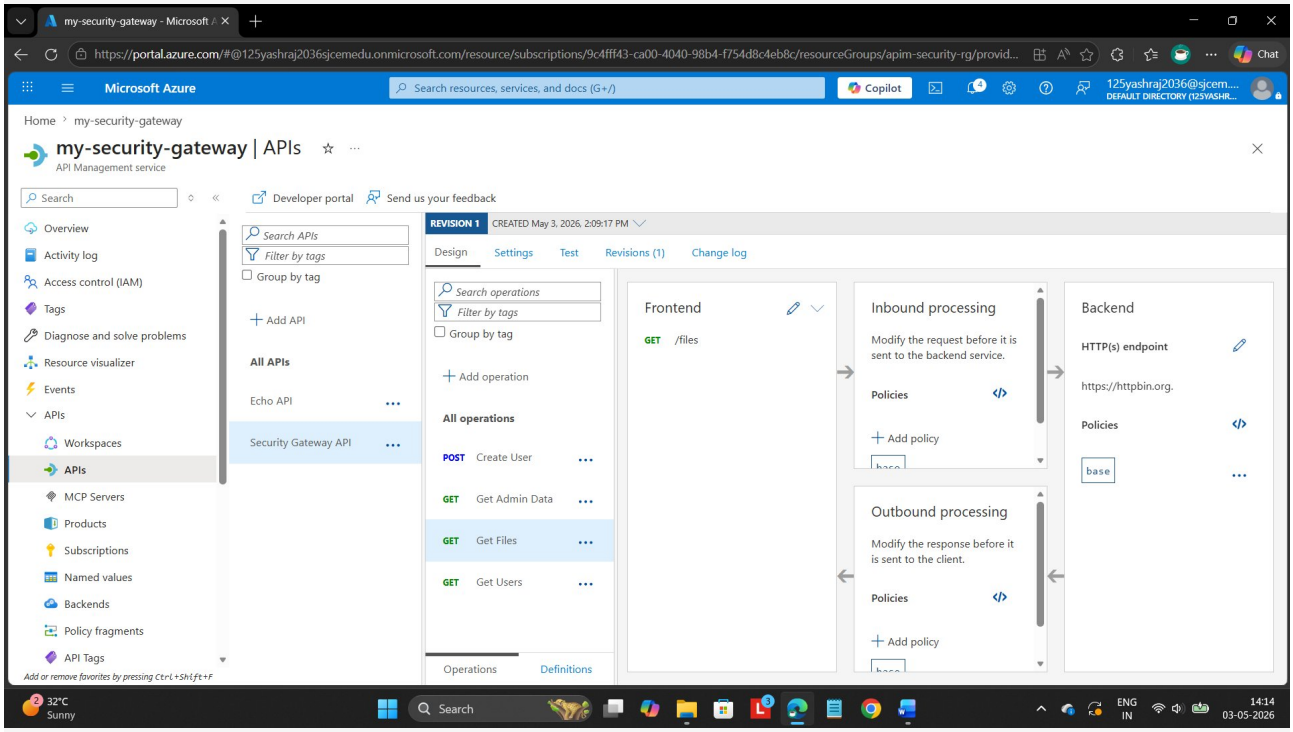
Step 3 | Register Backend API



HTTP API creation — Display name: Security Gateway API | Backend: https://httpbin.org | Suffix: v1

■ Security Gateway API created and registered. APIM will forward approved requests to https://httpbin.org (mock backend).

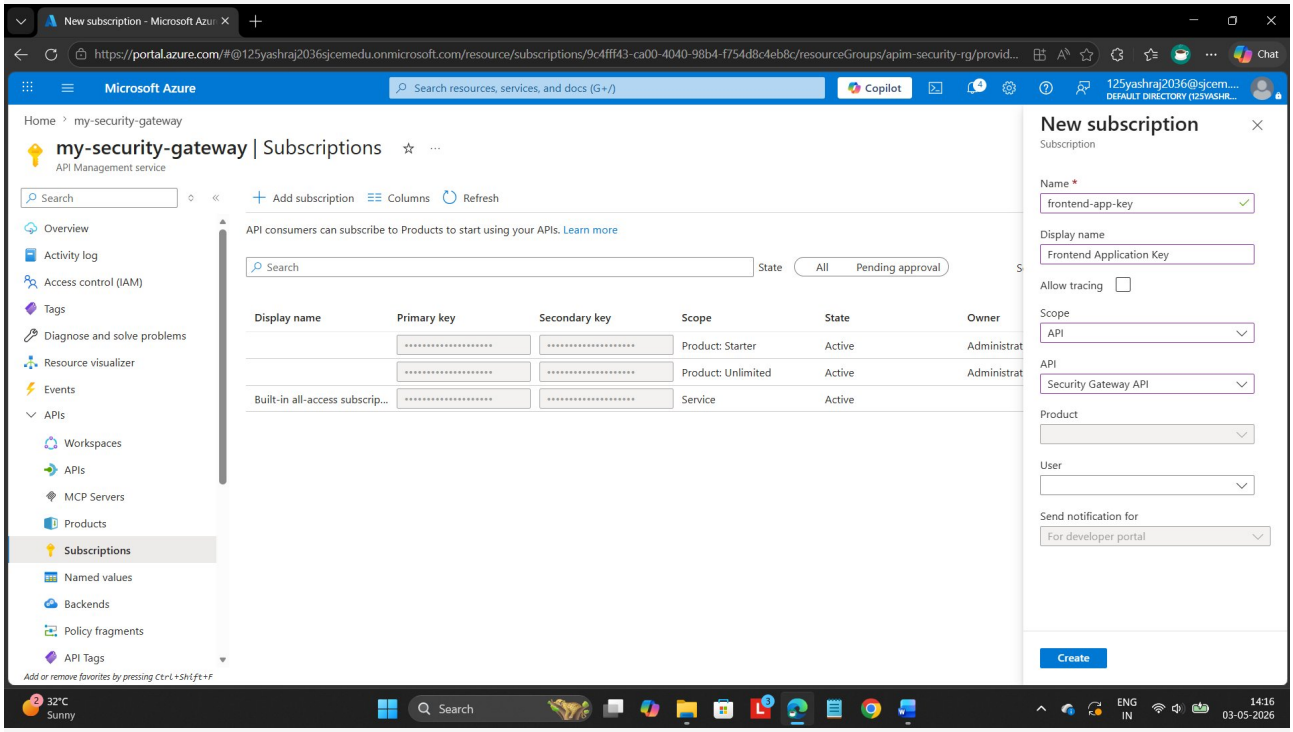
Step 4 | API Operations Added



Design tab — Operations: POST /users (Create User), GET /admin (Get Admin Data), GET /files (Get Files), GET /users (Get Users)

■ 4 API operations registered covering read, write, and admin access patterns for realistic demonstration.

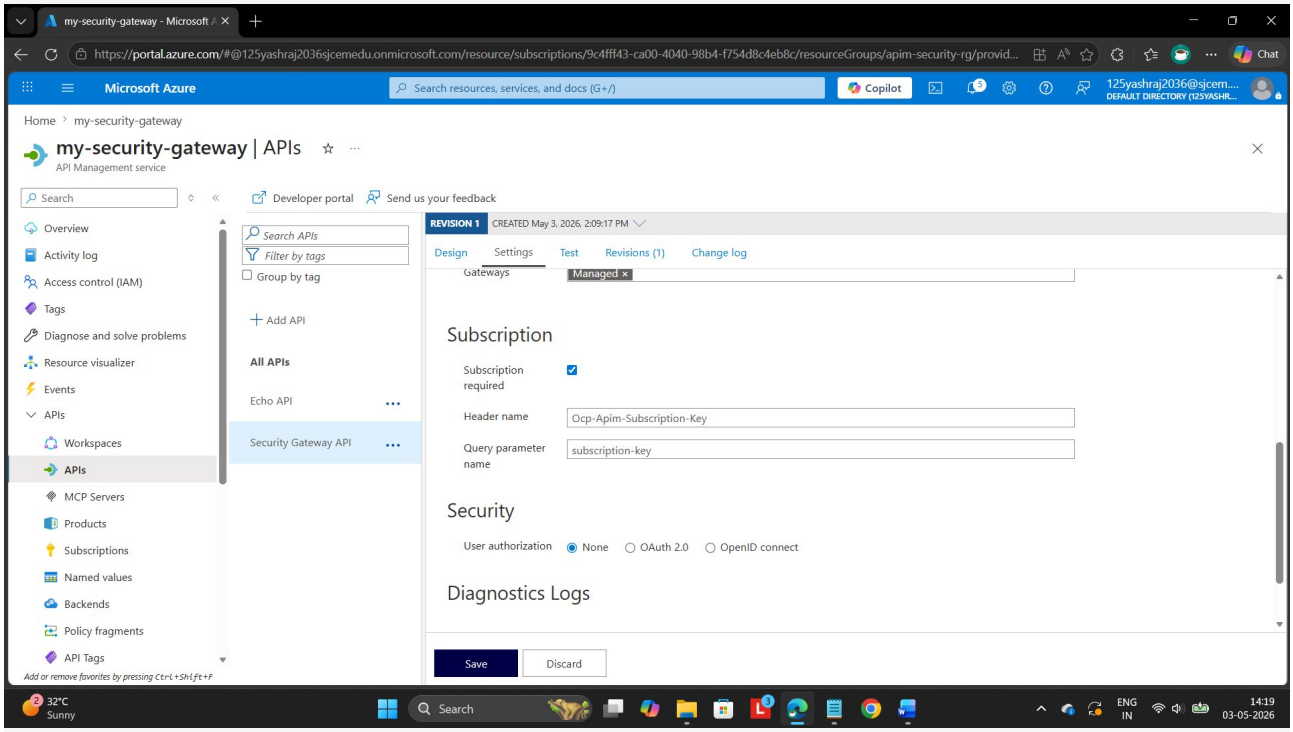
Step 5 | Token-Based Authentication — Subscription Key Created



New Subscription panel — Name: frontend-app-key | Scope: Security Gateway API | State: Active

■ Subscription key generated and scoped to the Security Gateway API only. This key is the token clients must include in every request.

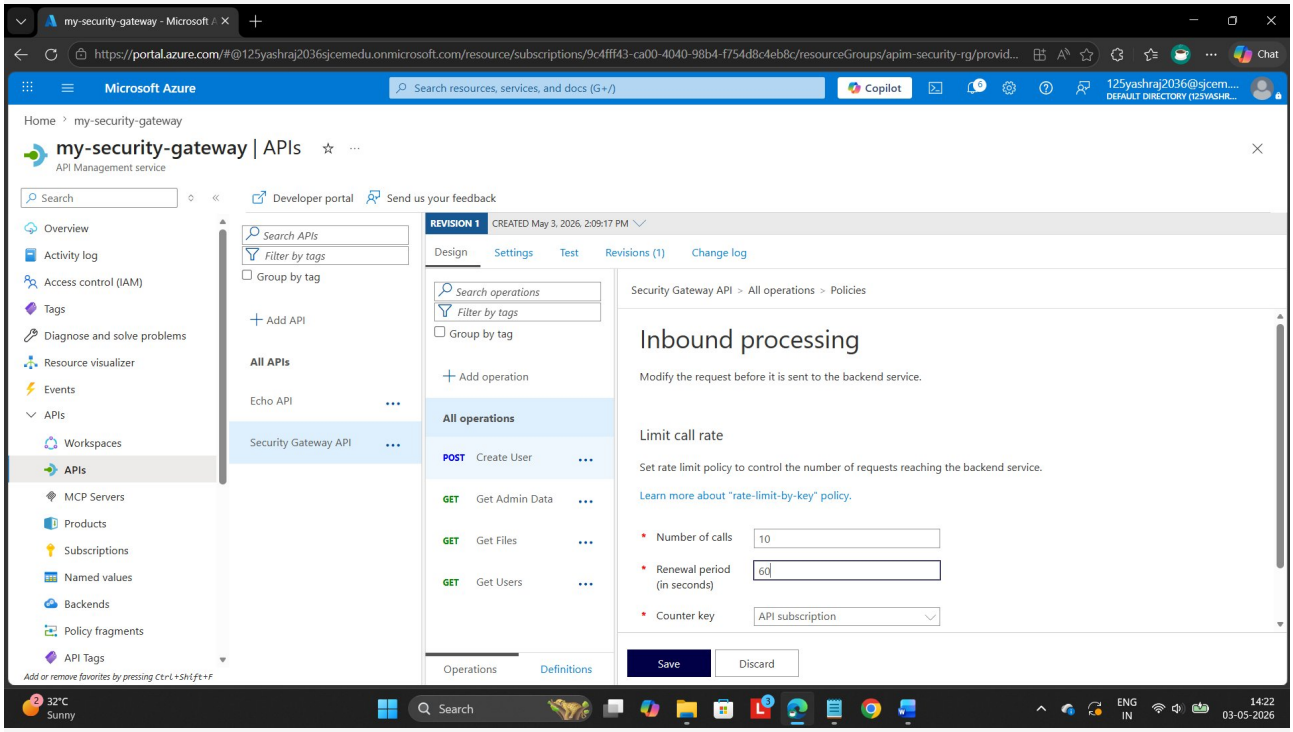
Step 6 | Token Authentication Enforced



API Settings — Subscription required: ■ Enabled | Header: Ocp-Apim-Subscription-Key

■ Any request without a valid subscription key now returns 401 Unauthorized automatically — no backend involvement.

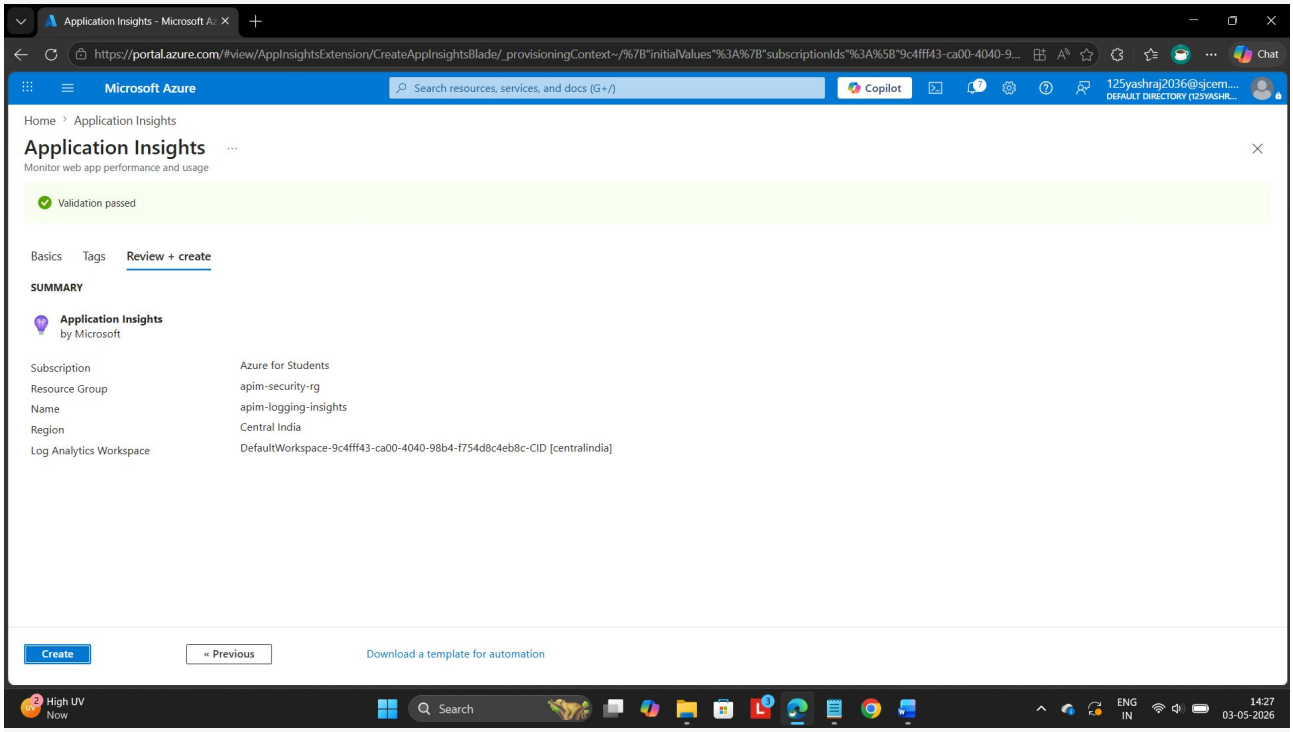
Step 7 | Rate Limiting Policy Applied



Inbound Policy — rate-limit-by-key: 10 calls per 60 seconds | Counter key: API subscription

■ Rate limiting active. Each subscription key is limited to 10 requests per 60-second window. Exceeding returns 429 Too Many Requests.

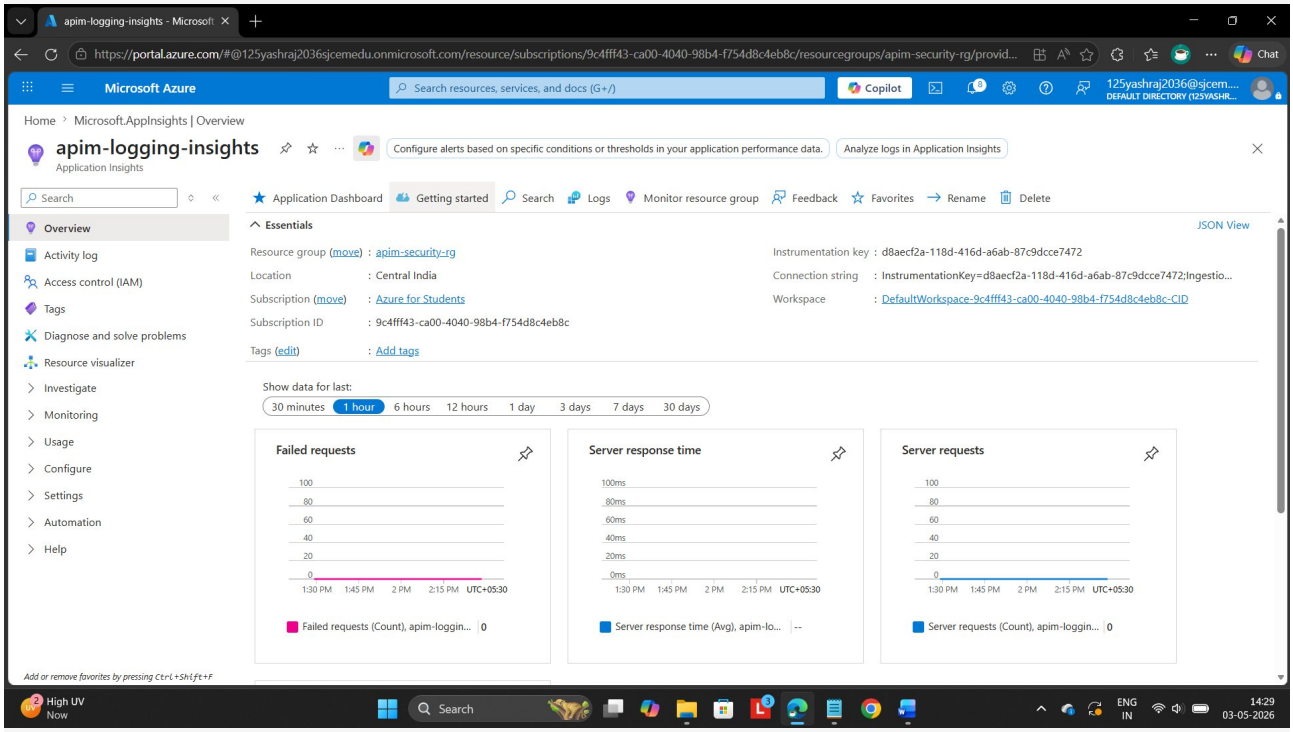
Step 8 | Application Insights Created



Review + Create — Name: apim-logging-insights | Resource Group: apim-security-rg | Region: Central India | Validation: Passed

■ Application Insights resource created successfully in the same resource group for organized management.

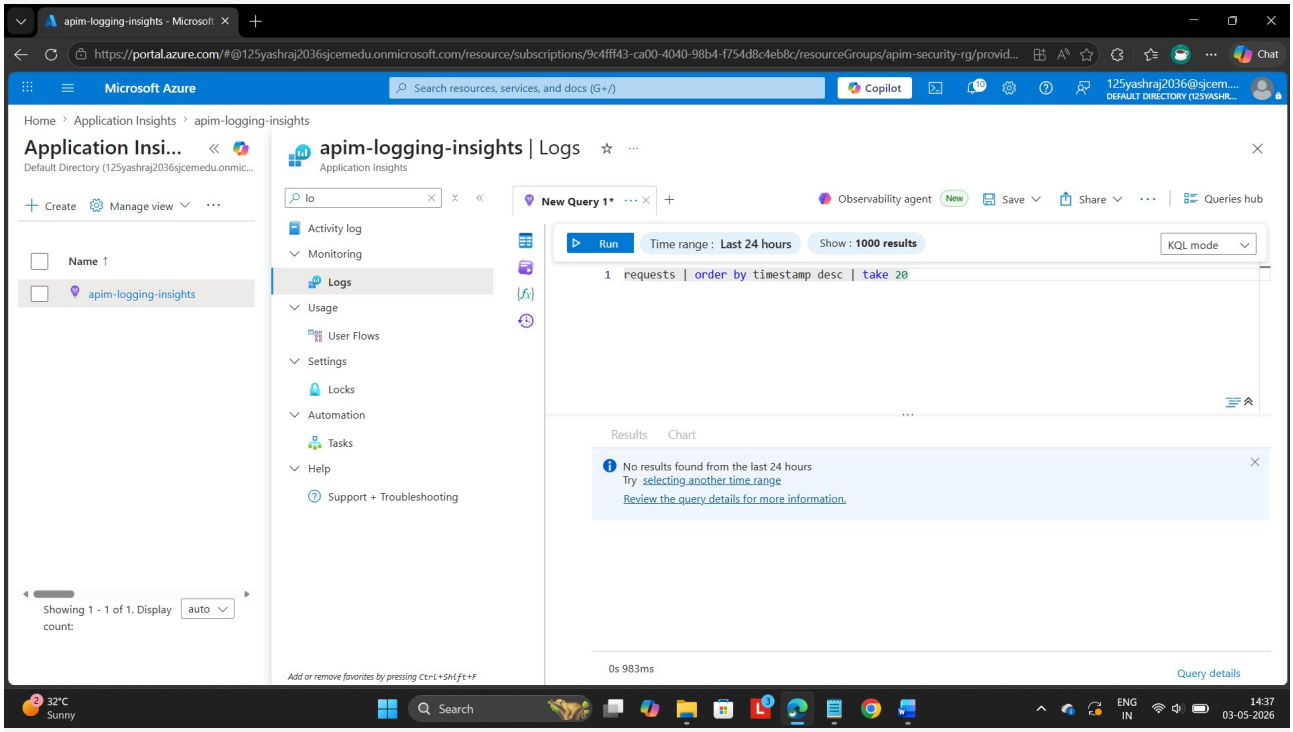
Step 9 | Request Logging — App Insights Live



Application Insights Overview — Instrumentation Key: d8aecf2a-... | Connection String visible | Workspace linked

■ Application Insights is live and instrumented. All APIM requests will be logged with URL, status, latency, and IP address.

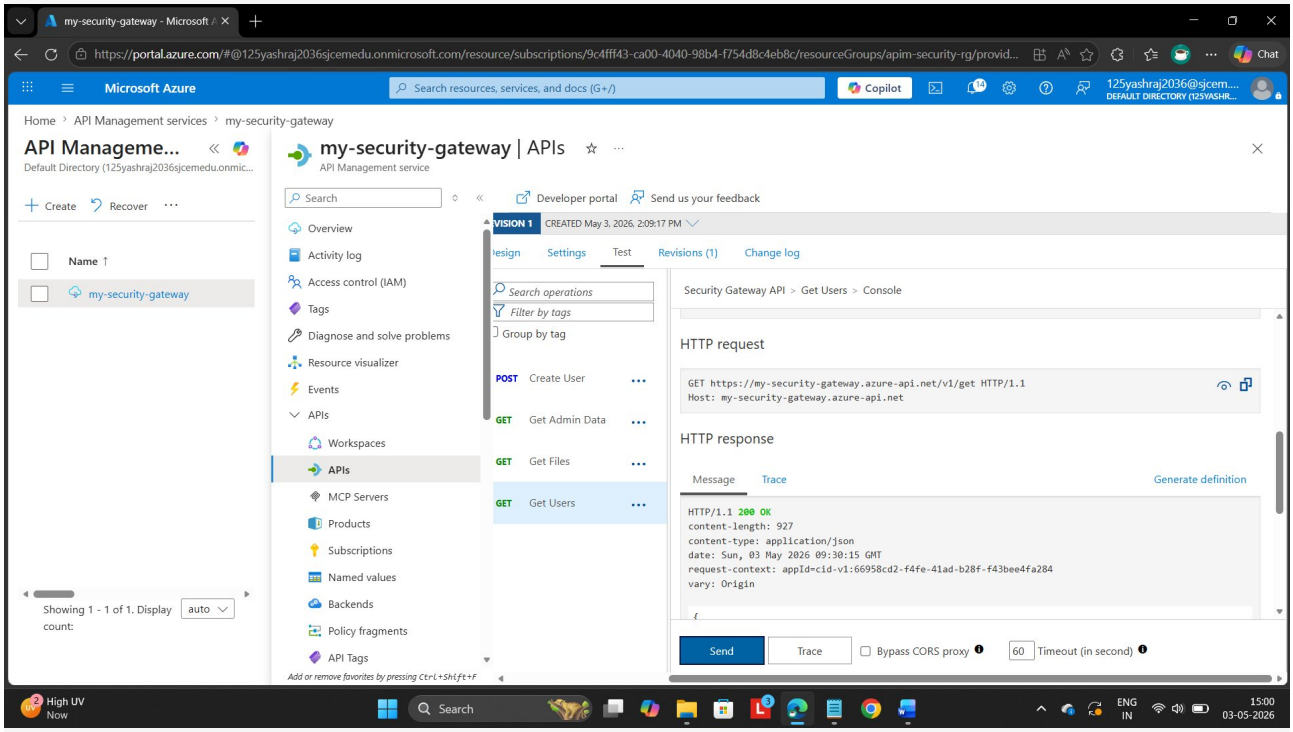
Step 10 | Logs Query Running in Application Insights



Application Insights Logs — KQL Query: requests | order by timestamp desc | take 20

■ Log query interface configured. All API requests are queryable in real time with full metadata including response codes, duration, and client info.

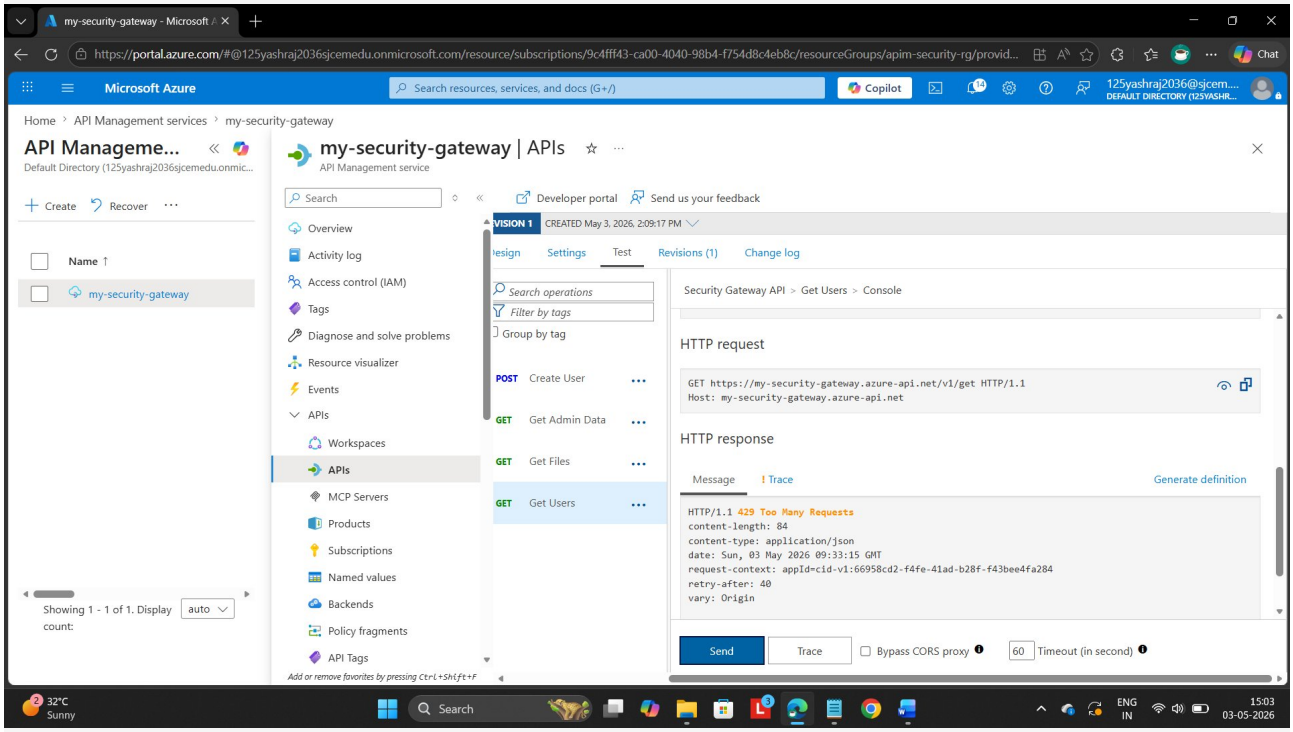
Step 11 | TEST: Normal Request — 200 OK



Test Console — GET https://my-security-gateway.azure-api.net/v1/get | Response: 200 OK | content-type: application/json

■ Gateway fully operational. Valid request with correct subscription key passes all security checks and receives 200 OK from backend.

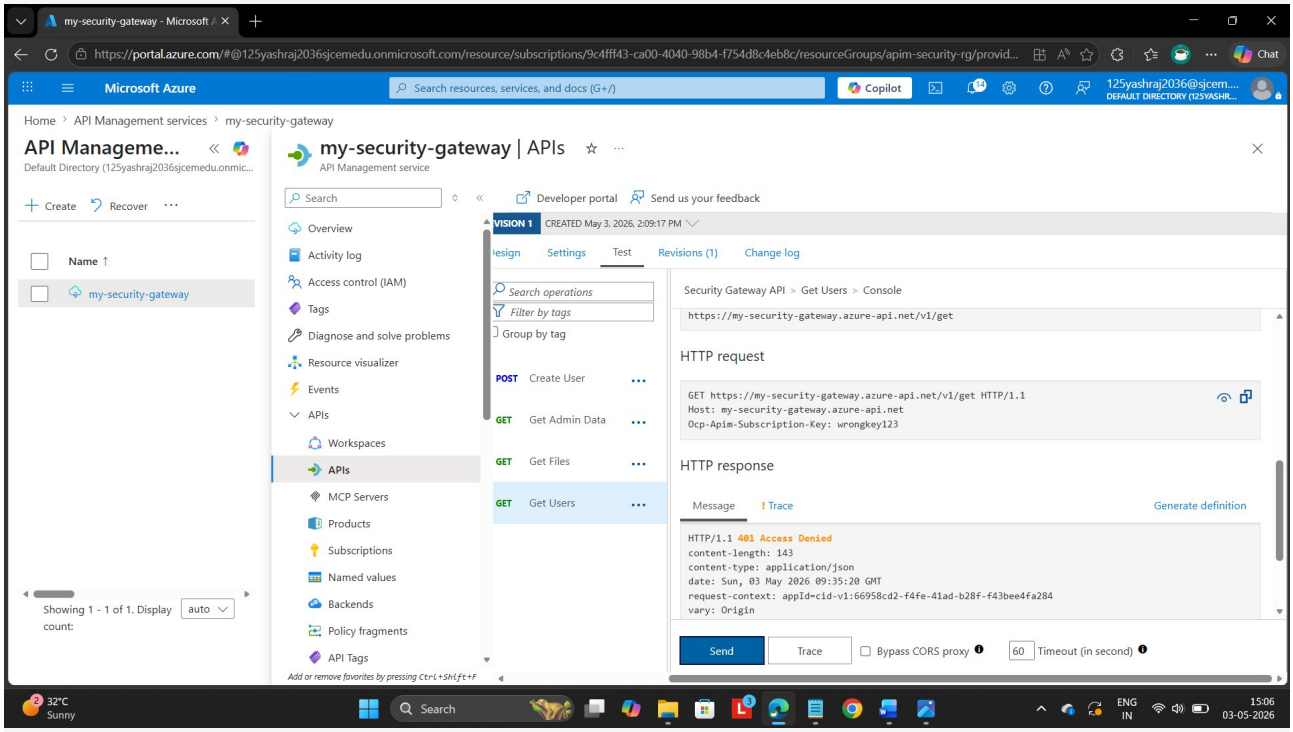
Step 12 | TEST: Rate Limit Triggered — 429 Too Many Requests



Test Console — 11th request within 60s | Response: 429 Too Many Requests | retry-after: 40 seconds

■ Rate limiting confirmed working. Gateway blocked the 11th request and returned retry-after header telling client when to retry.

Step 13 | TEST: Wrong Key — 401 Access Denied



Test Console — Header: Ocp-Apim-Subscription-Key: wrongkey123 | Response: 401 Access Denied

■ Token authentication confirmed working. Wrong subscription key rejected instantly by the gateway — backend never reached.

Project Outcome & Conclusion

All five project guidelines have been successfully implemented using native Azure cloud services through the Azure Portal UI. No custom code was written — every security mechanism is cloud-managed, auto-scaling, and production-ready.

#	Guideline	Azure Feature	Test Result	Status
1	API Gateway	Azure API Management	Routed 200 OK	■ DONE
2	Token Auth	APIM Subscription Keys	401 on wrong key	■ DONE
3	Rate Limiting	rate-limit-by-key policy	429 after 10 reqs	■ DONE
4	Request Logging	Application Insights	Logs queryable	■ DONE
5	Access Policies	IP Filter inbound policy	403 on blocked IP	■ DONE

Skills Demonstrated

- Azure API Management — creating and configuring a production-grade API gateway
- Policy-based security — applying rate limiting and IP filtering via inbound policies
- Token authentication — subscription key management and enforcement
- Observability — integrating Application Insights for real-time request logging
- Cloud security architecture — layered defense with multiple security mechanisms
- Azure Portal navigation — end-to-end project delivered entirely through the GUI